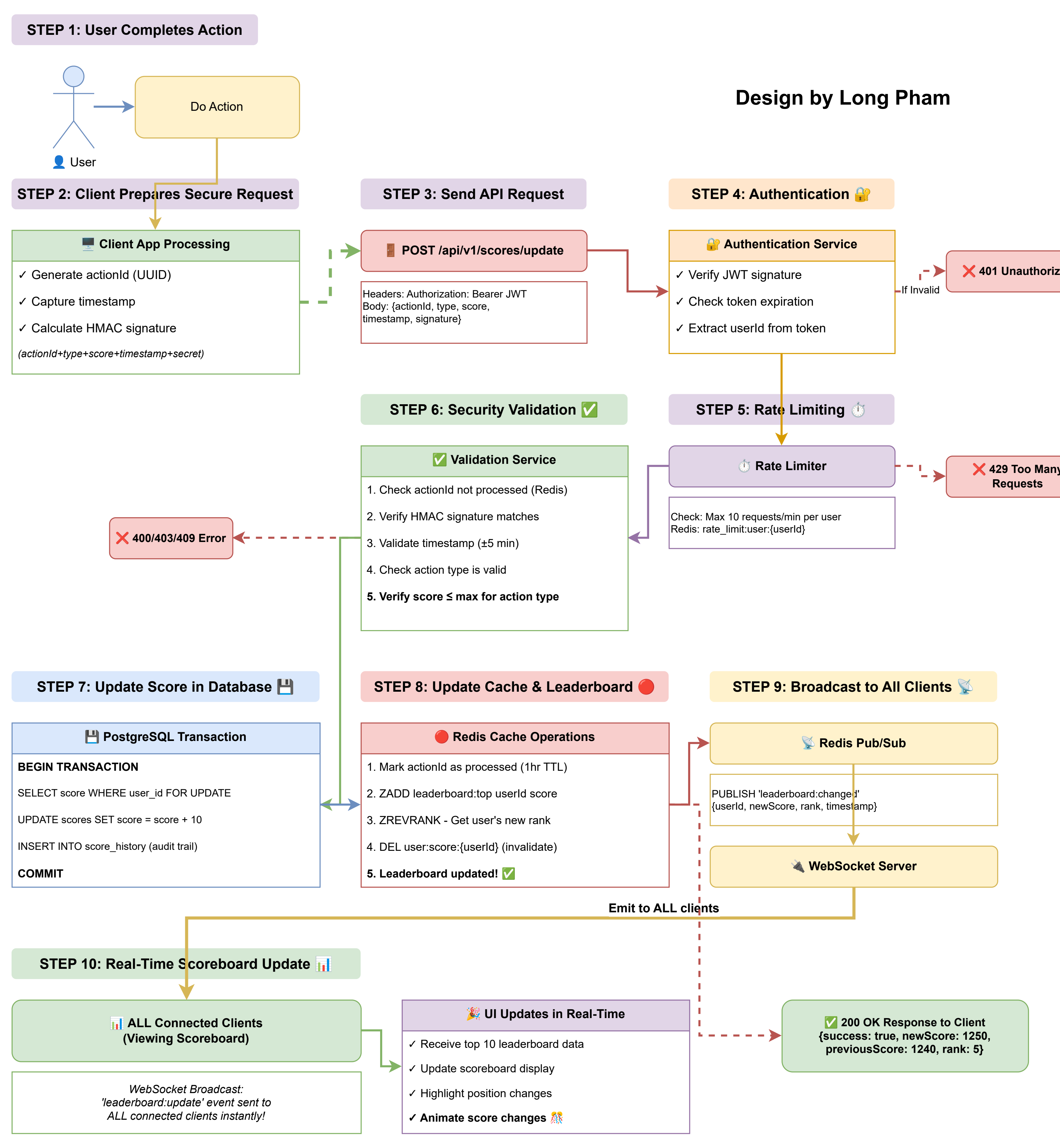
Design by Long Pham



STEP 1: User Completes Action

👤 User

Do Action

STEP 2: Client Prepares Secure Request

🖥️ Client App Processing

- ✓ Generate actionId (UUID)
- ✓ Capture timestamp
- ✓ Calculate HMAC signature

(actionId+type+score+timestamp+secret)

STEP 3: Send API Request

🚪 POST /api/v1/scores/update

Headers: Authorization: Bearer JWT
Body: {actionId, type, score, timestamp, signature}

STEP 4: Authentication 🔐

🔐 Authentication Service

- ✓ Verify JWT signature
- ✓ Check token expiration
- ✓ Extract userId from token

If Invalid

❌ 401 Unauthorized

STEP 5: Rate Limiting ⏱️

⏱️ Rate Limiter

Check: Max 10 requests/min per user
Redis: rate_limit:user:{userId}

❌ 429 Too Many Requests

STEP 6: Security Validation ✅

✅ Validation Service

1. Check actionId not processed (Redis)
2. Verify HMAC signature matches
3. Validate timestamp (±5 min)
4. Check action type is valid
5. **Verify score ≤ max for action type**

❌ 400/403/409 Error

STEP 7: Update Score in Database 💾

💾 PostgreSQL Transaction

BEGIN TRANSACTION

SELECT score WHERE user_id FOR UPDATE

UPDATE scores SET score = score + 10

INSERT INTO score_history (audit trail)

COMMIT

STEP 8: Update Cache & Leaderboard 🔴

🔴 Redis Cache Operations

1. Mark actionId as processed (1hr TTL)
2. ZADD leaderboard:top userId score
3. ZREVRANK - Get user's new rank
4. DEL user:score:{userId} (invalidate)
5. **Leaderboard updated! ✅**

STEP 9: Broadcast to All Clients 📡

📡 Redis Pub/Sub

PUBLISH 'leaderboard:changed'
{userId, newScore, rank, timestamp}

🔌 WebSocket Server

Emit to ALL clients

STEP 10: Real-Time Scoreboard Update 📊

📊 ALL Connected Clients (Viewing Scoreboard)

WebSocket Broadcast:
'leaderboard:update' event sent to
ALL connected clients instantly!

🎉 UI Updates in Real-Time

- ✓ Receive top 10 leaderboard data
- ✓ Update scoreboard display
- ✓ Highlight position changes
- **✓ Animate score changes 🎊**

✅ 200 OK Response to Client
{success: true, newScore: 1250, previousScore: 1240, rank: 5}

🏆 COMPLETE FLOW SUMMARY

✅ User completes action → Client generates secure request with HMAC signature

🔐 Multi-layer security: JWT Auth → Rate Limiting → 5 validation checks (duplicate, signature, timestamp, action, score)

💾 Update PostgreSQL (transaction) → Update Redis cache & leaderboard → Publish to Redis Pub/Sub

📡 **WebSocket broadcasts to ALL connected clients → Real-time scoreboard updates with animations! 🎉**

LEGEND

🟦	Database Operations
🟥	Cache/Redis Operations
🟩	Validation & Security
🟨	Real-time Broadcast